Web Application Basics

TryHackMe Learning & Practical Report

Prepared by: Kaustubh Ashish Bangar

Focus: Web Application Security / WAPT Foundations

Platform: TryHackMe — Cyber Security 101

Room: Web Application Basics

A structured technical report documenting the concepts studied and practical HTTP exercises completed in the Web Application Basics room.

Prepared for learning documentation, portfolio use, and cybersecurity training.

1. Introduction

A web application is a dynamic software program that operates on a web server and is accessed by users through a web browser over a network or the internet. Unlike static websites that merely display fixed HTML content, web applications allow for robust user interaction, data submission, backend processing, and database integration. Common examples include online banking portals, e-commerce platforms, social media networks, and internal enterprise dashboards.

In the context of Web Application Penetration Testing (WAPT), understanding the underlying mechanics of these applications is the most critical prerequisite. Before a security professional can identify, exploit, or mitigate vulnerabilities, they must possess a deep understanding of how clients and servers exchange data. The TryHackMe Web Application Basics module serves as this foundational layer. It details the architecture of modern web communication, focusing heavily on how web browsers construct requests, how servers parse those requests to deliver resources, and how modern web protocols maintain security and user state across multiple interactions.

This report comprehensively documents the theoretical concepts studied, the practical API manipulation exercises performed, the tools utilized, and the critical security takeaways derived from the module. The knowledge consolidated in this report establishes the necessary groundwork for advanced penetration testing topics, including reconnaissance, traffic interception via Burp Suite, and the exploitation of common web vulnerabilities.

2. Objective

The primary objective of this module and the resulting report is to establish a rock-solid foundation in web application communication protocols, architecture, and basic security mechanisms.

The specific learning and practical objectives include:

1. **Architecture Comprehension:** To understand the fundamental client-server model and the various components involved in delivering a modern web application.
2. **URL Anatomy:** To dissect the structure of a Uniform Resource Locator (URL) and understand the security implications of its components, particularly query strings.
3. **Protocol Distinction:** To differentiate between HTTP and HTTPS, understanding the necessity of encryption in transit.
4. **Message Structuring:** To study the exact composition of HTTP requests and responses, including request lines, headers, and message bodies.
5. **HTTP Methods:** To identify and utilize common HTTP methods (GET, POST, DELETE, OPTIONS) and understand their intended state-changing behaviors.
6. **Status Code Analysis:** To decode HTTP response status codes (1xx through 5xx) to quickly ascertain how a server processed a given request.
7. **Header Functions:** To analyze HTTP request and response headers, focusing on how they dictate content types and expose server metadata.
8. **State Management:** To understand how stateless HTTP communication utilizes cookies and sessions to maintain user state, alongside the security attributes (`Secure`, `HttpOnly`) that protect them.

9. **Security Policies:** To learn how defensive browser-side security headers, such as Content-Security-Policy (CSP) and HTTP Strict Transport Security (HSTS), mitigate client-side attacks.
10. **Practical Application:** To interact practically with application programming interfaces (APIs) by manually crafting GET, POST, and DELETE requests to read, modify, and destroy server-side resources.

3. Concept / Understanding

The theoretical portion of this module focused heavily on the mechanics of the HTTP protocol and how web applications process user-supplied data.

Web Application Architecture The core architecture of a web application relies on a continuous dialogue between a client and a server. The client, typically a modern web browser, acts as the user interface, responsible for rendering HTML, CSS, and executing client-side JavaScript. It constructs and formats requests based on user actions. The web server receives these requests, processes them through application logic, interacts with backend databases if necessary, and formulates an appropriate response. Additionally, this ecosystem often includes a Web Application Firewall (WAF), which sits in front of the server to inspect incoming HTTP traffic and block known malicious payloads or attack patterns. Understanding this multi-layered architecture is vital for WAPT, as each component—from the browser's rendering engine to the backend database—represents a unique attack surface.

URL Structure and Security A Uniform Resource Locator (URL) is the standard method for identifying and locating resources on the internet. A standard URL consists of several distinct parts:

- **Scheme:** Defines the protocol used (e.g., `http://` or `https://`).
- **Hostname/Domain:** Specifies the server's address (e.g., `[www.example.com](https://www.example.com)`).
- **Path:** Indicates the specific resource or directory on the server (e.g., `/api/users`).
- **Query String:** Appended after a `?`, this component carries parameters in key-value pairs (e.g., `?id=1&role=admin`).

From a penetration testing perspective, query strings are critical because they represent direct, user-controlled input. Attackers frequently manipulate these parameters to test for vulnerabilities like SQL Injection, Cross-Site Scripting (XSS), or Insecure Direct Object References (IDOR). Additionally, the module covered the concept of *typosquatting*, where attackers register domain names that are visually similar to legitimate domains (e.g., `[www.exampIe.com](https://www.exampIe.com)`) to deceive users and capture credentials or distribute malware.

HTTP and HTTPS Hypertext Transfer Protocol (HTTP) is the foundational plaintext protocol for web communication. Because HTTP transmits data—including passwords and session tokens—in clear text, it is highly susceptible to interception via Man-in-the-Middle (MitM) attacks. HTTPS (HTTP Secure) resolves this by wrapping the standard HTTP communication within a secure, encrypted Transport Layer Security (TLS) tunnel.

HTTP Requests and Responses HTTP is a request-response protocol. A client sends an HTTP request, and the server returns an HTTP response.

- **HTTP Request Structure:** Begins with a request line containing the method, the target path, and the HTTP version (e.g., `GET /index.html HTTP/1.1`). This is followed by request headers, an empty line, and optionally, a message body (used primarily in POST or PUT requests to transmit data).
- **HTTP Response Structure:** Begins with a status line containing the HTTP version, a numerical status code, and a brief text explanation (e.g., `HTTP/1.1 200 OK`). This is followed by response headers, an empty line, and the response body containing the requested resource (like HTML code or JSON data).

HTTP Methods

Methods indicate the desired action to be performed on the target resource:

- **GET:** Used exclusively to retrieve data from the server. It should not cause any state changes.
- **POST:** Used to submit data to the server, frequently resulting in a change in state or the creation of a new resource (e.g., submitting a login form or updating a user profile).
- **DELETE:** Instructs the server to remove the specified resource.
- **OPTIONS:** Used by the client to discover which HTTP methods the server permits for a specific endpoint.

HTTP Headers

Headers are key-value pairs that transmit metadata about the request or response.

- **Request Headers:** Includes the mandatory `Host` header (which identifies the target domain), `User-Agent` (identifying the client software), and `Content-Type` (which tells the server how to parse the request body, such as `application/x-www-form-urlencoded` for standard forms or `application/json` for APIs).
- **Response Headers:** Provide information back to the client. The `Server` header, for instance, might disclose the underlying server software (e.g., Apache or Nginx) and its version, which is useful during the reconnaissance phase of a WAPT assessment to identify vulnerable software versions.

HTTP Status Codes Status codes provide immediate feedback on the result of an HTTP request:

- **2xx (Success):** Indicates the request was successfully received and processed (e.g., `200 OK`).
- **3xx (Redirection):** Indicates further action needs to be taken, usually instructing the browser to navigate to a different URL.
- **4xx (Client Error):** Indicates the request contains bad syntax or cannot be fulfilled. Examples include `401 Unauthorized`, `403 Forbidden` (lack of permissions), and `404 Not Found` (resource does not exist).
- **5xx (Server Error):** Indicates the server failed to fulfill an apparently valid request due to an internal issue (e.g., `500 Internal Server Error`).

Cookies and Sessions

HTTP is fundamentally a stateless protocol, meaning the server treats every request as completely independent. To build applications that require persistent state—such as staying logged into a shopping site—developers use sessions and cookies. When a user logs in, the server generates a unique session identifier and sends it to the client via the `Set-Cookie` response header. The browser then automatically includes this cookie in all subsequent requests to that domain, allowing the server to recognize the user.

To secure these cookies against theft, several attributes are used:

- **Secure:** Instructs the browser to only transmit the cookie over encrypted HTTPS connections, protecting it from network interception.
- **HttpOnly:** Instructs the browser to prevent client-side JavaScript from accessing the cookie. This heavily mitigates the risk of session hijacking via Cross-Site Scripting (XSS) attacks.

Security Headers Modern web browsers enforce security policies defined by the server through specific HTTP response headers:

- **Content-Security-Policy (CSP):** A highly effective header that restricts the locations from which a browser is permitted to load resources (like scripts, images, and stylesheets). For example, a `script-src` directive can prevent the execution of malicious scripts injected by an attacker.
- **HTTP Strict Transport Security (HSTS):** Forces the browser to communicate with the server exclusively over HTTPS for a specified duration, preventing downgrade attacks.
- **X-Content-Type-Options: nosniff:** Prevents the browser from trying to guess (sniff) the MIME type of a file, forcing it to strictly adhere to the `Content-Type` declared by the server.

4. Practical Work

The practical application phase of the module required interacting directly with a simulated backend API to retrieve, modify, and delete data using different HTTP methods. This effectively transitioned the theoretical protocol knowledge into practical penetration testing skills.

- **GET Request (Data Retrieval):** The first objective was to query the application's user database. I crafted and sent an HTTP `GET` request to the `/api/users` endpoint. The server successfully processed the request, verified that no state change was requested, and returned a `200 OK` status code along with the requested list of users in the response body. This validated the standard use-case of the GET method for safe data retrieval.
- **POST Request (Data Modification):** The second objective involved causing a state change on the server. The task was to update the profile information for a specific user (User ID 2, "Bob") and change his country data from the UK to the US. I constructed an HTTP `POST` request directed at the `/api/user/2` endpoint. Crucially, I included the application data `country=US` within the request body. The server parsed this body, updated the backend database, and returned a success confirmation. This demonstrated how client-supplied input directly influences server-side state.

- **DELETE Request (Data Removal):** The final practical objective was to destroy a server-side resource. I sent an HTTP `DELETE` request targeting the `/api/user/1` endpoint. The server executed the destructive operation and removed the specified user from the database.

5. Tools Used

- **TryHackMe Platform:** Provided the hosted, isolated cloud environment and the vulnerable API endpoints required for safe testing.
- **Web Browser:** Used as the primary HTTP client to navigate the module and interact with the lab environment.
- **In-Browser HTTP Client/Developer Tools:** Utilized to manually craft, inspect, modify, and issue the exact HTTP requests (GET, POST, DELETE) required to solve the API challenges.

6. Evidence and Screenshots

All relevant visual evidence of the completed exercises is stored in the designated submission directory.

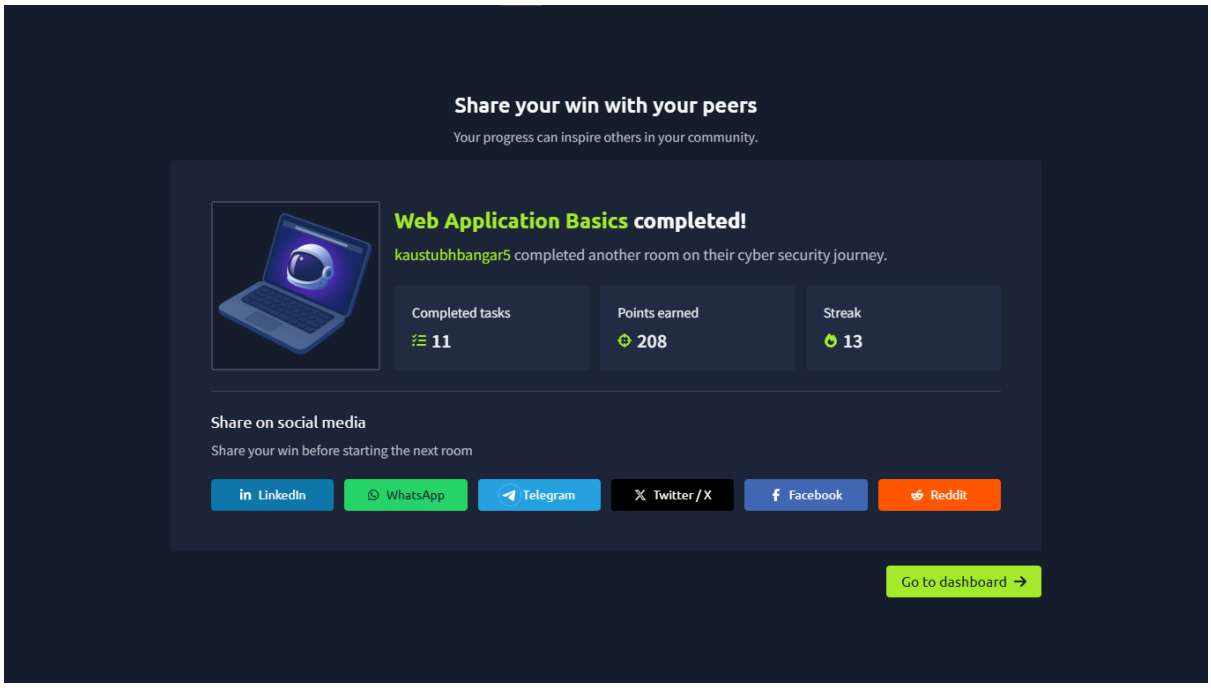


Fig. 1.1: TryHackMe room completion certificate and dashboard progress.

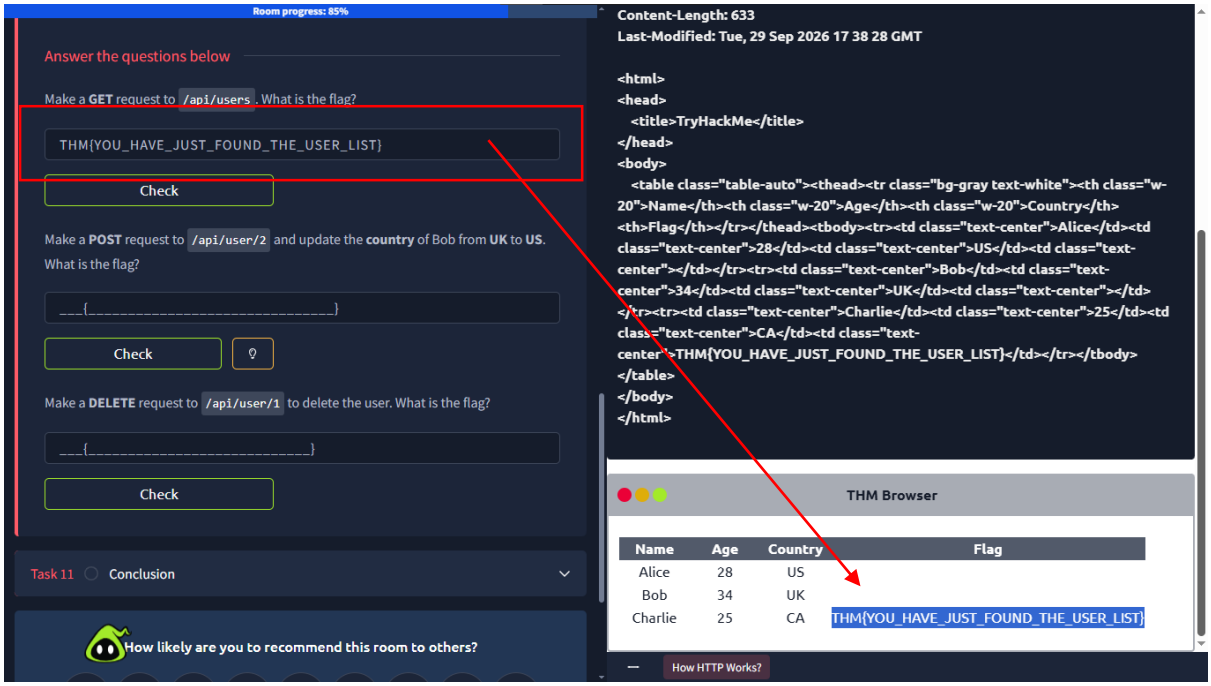


Fig. 1.2: Screenshot showing the successful construction and server response of the GET request to `/api/users`.

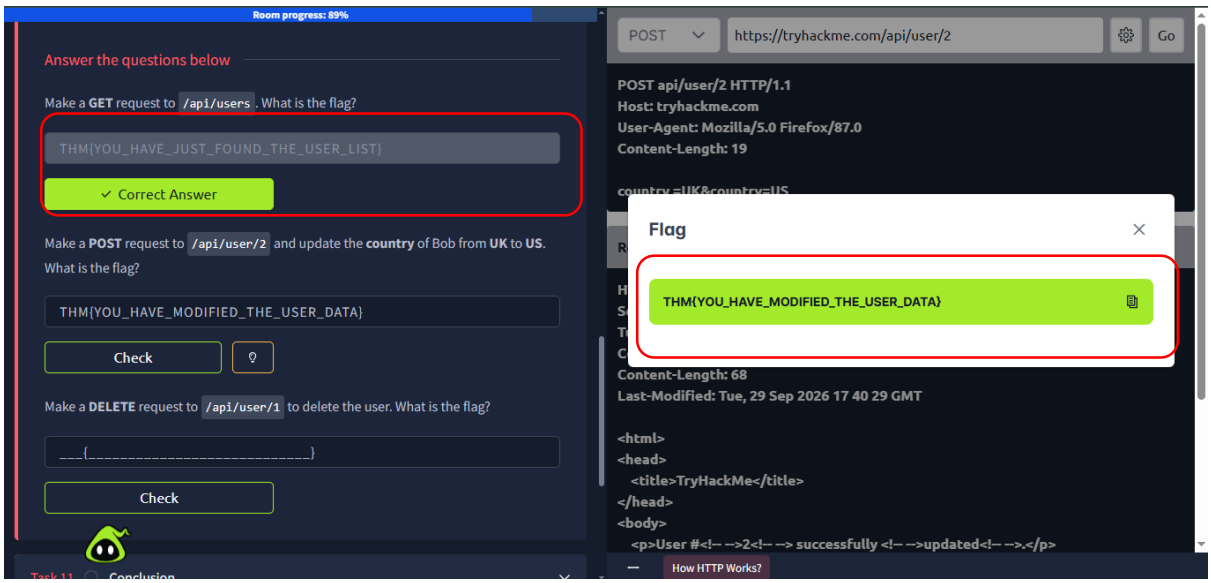


Fig. 1.3: Screenshot capturing the POST request payload (`country=US`) and the subsequent successful state-change response from `/api/user/2`.

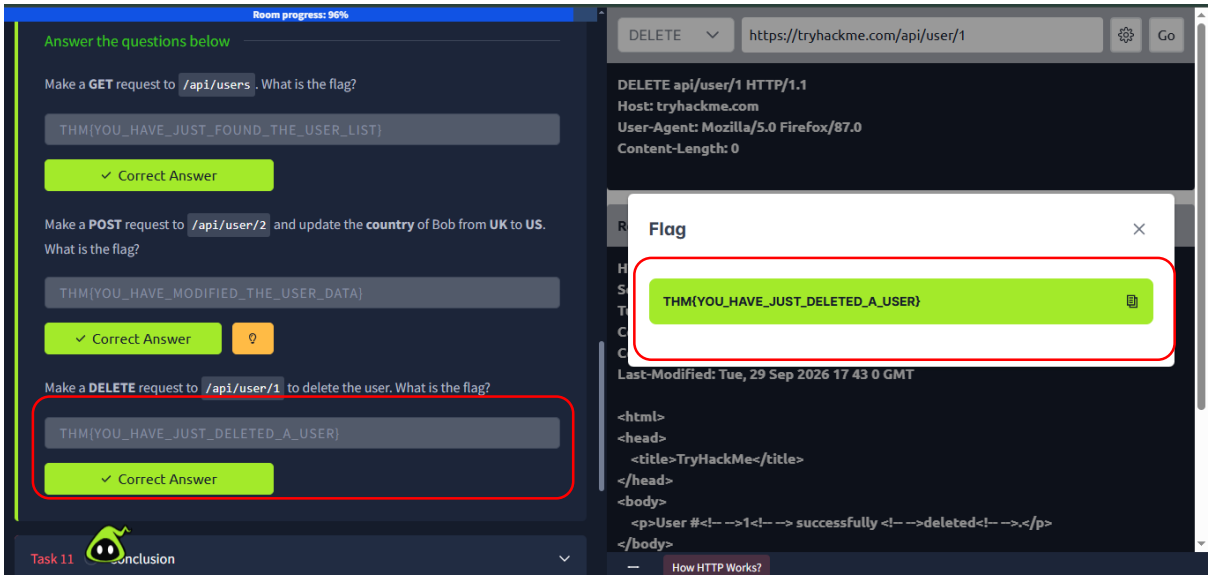


Fig. 1.4: Screenshot documenting the successful DELETE request executed against `/api/user/1`.

7. Results

The Web Application Basics module was completed successfully. I gained a concrete, functional understanding of how web browsers and servers interact via the HTTP protocol. During the practical lab, I successfully enumerated API endpoints and successfully manipulated application data by retrieving user lists via GET, forcing backend database modifications via POST, and permanently removing resources via DELETE.

8. Key Learnings

Completing this module yielded several critical insights regarding web application security:

- **The Foundation of Security:** Web application penetration testing relies entirely on understanding standard, secure web communication. A tester must understand how an application is *supposed* to function normally before they can identify where trust boundaries break down or where security controls fail.
- **Never Trust Client Input:** URL parameters, query strings, and HTTP request bodies are entirely controlled by the end-user (or attacker). The application server must never blindly trust this data; it requires rigorous server-side input validation and sanitization.
- **Authorization is Distinct from Protocol Methods:** While HTTP methods (like DELETE) communicate the *intent* to perform a specific action, the server is solely responsible for verifying if the requesting user actually has the authorization and privileges required to perform that destructive action.
- **Information Leakage in Responses:** Analyzing HTTP status codes and response headers provides invaluable intelligence. Differences between a `401`, `403`, and `404` error can allow an attacker to enumerate hidden directories, while the `Server` header can easily leak vulnerable software versions.

- **Defense-in-Depth Mechanisms:** Setting `Secure` and `HttpOnly` flags on session cookies, combined with implementing strict browser security headers like CSP and HSTS, significantly reduces the attack surface and mitigates the impact of vulnerabilities like XSS, even if underlying flaws exist in the application code.

9. Conclusion

The TryHackMe Web Application Basics room provided a robust, comprehensive introduction to the architecture of the modern web. By deconstructing URLs, thoroughly analyzing HTTP request and response structures, and practically manipulating API endpoints, I shifted my perspective from viewing a website as a simple graphical interface to understanding it as a complex series of structured data exchanges.

The practical exercises executing GET, POST, and DELETE requests solidified how user-supplied data interacts with backend systems and highlighted the vital necessity of strict server-side authorization controls. This foundational knowledge of parameters, headers, cookies, and HTTP methods is the direct prerequisite for utilizing professional proxy tools like Burp Suite, allowing for advanced request interception, modification, and the systematic discovery of vulnerabilities in subsequent WAPT phases.

10. References

1. TryHackMe - Web Application Basics: https://tryhackme.com/room/webapplicationbasics
2. WAPT Phase 1 Task Instructions - IqraSec Academy